ASSIGNMENT – 4 RELATIONAL DATABASE VS. NOSQL DATABASE

DATABASE MANAGEMENT SYSTEM

ALLIANCE UNIVERSITY

Department of Computer Applications

SUMMITED BY: KUNGUMA KEERTHANA M

REGISTER NO: 2511021240051

COURSE: BCA

YEAR: 2ND YEAR

PROJECT TITLE: Online Food Delivery System

1. Aim

To compare relational databases and NoSQL databases based on scalability, consistency, performance, and use cases, and to understand why organizations select different database technologies according to their requirements.

2. Introduction

A database is used to store, organize, manage, and retrieve information. Organizations choose different database technologies depending on their application requirements, data structure, performance needs, and expected growth.

The two major database types are relational databases and NoSQL databases.
A relational database stores data in tables containing rows and columns. A NoSQL database supports different data models, such as documents, key-value pairs, columns, and graphs.

3. Relational Database

A relational database organizes data into tables and connects them using primary keys and foreign keys. It generally uses Structured Query Language (SQL) to manage and retrieve data.

Examples: MySQL, PostgreSQL, Oracle Database, and Microsoft SQL Server.

Example in the Online Food Delivery System:

The database can contain separate tables for Customer, Restaurant, Food_Item, Orders, Payment, and Delivery. These tables are connected through keys to maintain relationships between the data.

4. NoSQL Database

A NoSQL database stores data using models such as documents, key-value pairs, wide-column stores, or graphs. It is useful when applications need flexible data structures, large-scale distribution, or specialized data access patterns.

Examples: MongoDB, Apache Cassandra, Redis, and Neo4j.

Example in the Online Food Delivery System:

A document database such as MongoDB could store a customer's order details, including food items, delivery instructions, and order status, in a document. This can be useful when the application needs to handle flexible order information.

5. Comparison Between Relational and NoSQL Databases

Feature	Relational Database	NoSQL Database
Data structure	Tables with rows and columns	Documents, key-value pairs, columns, or graphs
Schema	Usually predefined	Often flexible, depending on the database
Scalability	Commonly scales vertically; horizontal scaling is also possible	Many systems support horizontal scaling across multiple servers
Consistency	Strong consistency and transactions are commonly supported	Consistency depends on the system and its configuration
Performance	Efficient for structured data, joins, and complex queries	Efficient for suitable access patterns involving large or flexible datasets
Relationships	Supports relationships through keys and joins	Relationships may be embedded, referenced, or represented through specialized models
Transactions	Supports transactions; capabilities vary by product	Transaction support varies; some systems support multi-record transactions
Data redundancy	Normalization helps reduce unnecessary duplication	Some designs intentionally duplicate data to improve read performance
Best suited for	Structured data and applications requiring reliable relationships	Flexible, distributed, or specialized high-volume workloads
Examples	MySQL, PostgreSQL, Oracle	MongoDB, Cassandra, Redis, Neo4j

6. Scalability

Scalability is the ability of a database to handle increasing amounts of data and user requests.

Relational databases: They often scale vertically by increasing the CPU, memory, or storage of a server. Some relational systems also support horizontal scaling through replication, partitioning, or distributed architectures.

NoSQL databases: Many NoSQL systems are designed to scale horizontally by distributing data and workloads across multiple servers.

Example: An online food delivery application experiencing a large increase in orders during a festival may use a distributed database to handle high traffic and growing data volumes.

7. Consistency

Consistency means ensuring that database operations maintain valid and reliable data.

Relational databases: They commonly support ACID transactions, which help ensure that related operations are completed reliably.

NoSQL databases: Their consistency guarantees vary. Some provide strong consistency, while others offer configurable or eventual consistency depending on the product and operation.

Example: In the Online Food Delivery System, payment records and order status changes must remain reliable. A database with suitable transaction guarantees can help prevent incorrect payment and order records.

8. Performance

Performance refers to how quickly a database can read, write, update, and retrieve information.

Relational databases: They are suitable for structured information, complex queries, joins, and transactional operations.

NoSQL databases: They can provide efficient performance for particular access patterns, such as retrieving documents, caching frequently accessed values, or processing distributed workloads.

Actual performance depends on database design, indexing, workload, hardware, and configuration.

Example: A relational database can efficiently retrieve an order together with its customer and payment details. A key-value database such as Redis can be used to cache frequently accessed restaurant listings or menu information.

9. Real-World Examples

9.1 Banking System — Relational Database

Banking systems need reliable transactions when transferring money between accounts. Relational databases are suitable for such operations because they support transactional guarantees and structured records.

9.2 Social Media — NoSQL Database

Social media applications handle large volumes of posts, interactions, and changing data structures. NoSQL databases can be useful for particular workloads that require flexible data models and distributed scaling.

9.3 E-Commerce — Combination of Databases

An e-commerce application may use a relational database for orders and payments, a document database for flexible product information, and a key-value store for caching and sessions.

This demonstrates that organizations can use multiple database technologies within the same application.

9.4 Online Food Delivery System

For the proposed Online Food Delivery System, a relational database such as MySQL or PostgreSQL is a suitable primary choice because the project has structured tables and clear relationships.

It can manage customers, addresses, restaurants, food items, orders, payments, and delivery partners using primary keys, foreign keys, and transactions.

A NoSQL database could be added if the application later requires flexible event data, large-scale delivery tracking information, or other specialized workloads.

10. Why Organizations Choose Different Database Technologies

Organizations select database technologies based on several factors:

1. **Data structure:** Structured and strongly related data often suits relational databases, while flexible or varied data may suit NoSQL.
2. **Consistency requirements:** Financial transactions and order processing may require strong transactional guarantees.
3. **Scalability:** Applications expecting rapid growth may need distributed database capabilities.
4. **Performance:** The database must efficiently support the application's most common queries and operations.
5. **Flexibility:** Applications with frequently changing data structures may benefit from flexible schemas.
6. **Cost and maintenance:** Organizations consider infrastructure, licensing, administration, backup, and operational complexity.
7. **Application requirements:** The selected database must match the application's workload, security needs, and reliability requirements.

No single database technology is best for every situation. The appropriate choice depends on the application's specific requirements.

11. Result

Relational and NoSQL databases were compared based on scalability, consistency, performance, data structure, and use cases. Real-world examples demonstrated how different organizations choose database technologies according to their application requirements.

12. Conclusion

Relational databases are well suited for structured data, relationships, and transactional applications. NoSQL databases provide alternative data models and can support flexible schemas and distributed workloads. Both technologies have advantages and limitations.

For the proposed Online Food Delivery System, a relational database is a suitable primary choice because it maintains clear relationships between customers, orders, food items, payments, and deliveries. Additional NoSQL technologies may be introduced if future requirements justify them.